

Fəsil 9. Unsupervised Learning Texnikaları

Bu gün maşın öyrənmə tətbiqlərinin əksəriyyəti Supervised Learning (nəzarətli öyrənmə) əsasında olsa da (və nəticədə, investisiyaların çoxu bu sahəyə yönəlsə də), mövcud məlumatların böyük əksəriyyəti etiketsizdir: bizdə giriş xüsusiyyətləri X var, lakin etikətlər y mövcud deyil. Kompüter alimi Yann LeCun məşhur bir ifadəsində demişdi ki, "əgər intellekt tort olsaydı, Unsupervised Learning tortun özü, Supervised Learning tortun üzərindəki qıllazur, Reinforcement Learning isə tortun üzərindəki albalı olardı." Başqa sözlə, Unsupervised Learning sahəsində biz hələ yeni-yeni dadmağa başladığımız böyük bir potensial var.

Tutaq ki, istehsal xəttindəki hər bir məhsulun bir neçə şəklini çəkən və hansı məhsulların qüsurlu olduğunu aşkar edən bir sistem yaratmaq istəyirsiniz. Avtomatik olaraq şəkil çəkən bir sistem yaratmaq olduqca asandır və bu, sizə hər gün minlərlə şəkil verə bilər. Sonra bir neçə həftə ərzində kifayət qədər böyük bir dataset qura bilərsiniz. Amma gözləyin, heç bir etiket yoxdur! Əgər bir məhsulun qüsurlu olub-olmadığını proqnozlaşdıracaq adi bir binary classifier (ikili təsnifatçı) öyrətmək istəyirsinizsə, hər bir şəkli ayrı-ayrılıqda "qüsurlu" və ya "normal" olaraq etikətləməli olacaqsınız. Bu, ümumiyyətlə, insan ekspertlərin bütün şəkilləri əl ilə nəzərdən keçirməsini tələb edəcək. Bu, uzun, bahalı və yorucu bir işdir, buna görə də adətən yalnız mövcud şəkillərin kiçik bir alt çoxluğu üzərində həyata keçirilir. Nəticədə, etikətlənmiş dataset kifayət qədər kiçik olacaq və classifier-in performansы məyusedici olacaq. Üstəlik, şirkət məhsullarına hər hansı bir dəyişiklik etdikdə, bütün proses sıfırdan başlamalı olacaq. Əgər alqoritm insanların hər bir şəkli etikətləməsini tələb etmədən, sadəcə olaraq etiketsiz məlumatlardan

istifadə edə bilsəydi, nə qədər gözəl olardı? Unsupervised Learning-ə xoş gəlmisiniz.

Fəsil 8-də ən çox yayılmış Unsupervised Learning tapşırığına – dimensionality reduction (ölçülülüyün azaldılması) baxdıq. Bu fəsildə daha bir neçə Unsupervised tapşırığa nəzər salacağıq:

Clustering

Məqsəd oxşar instansları birlikdə *clusters* (qruplar) halında qruplaşdırmaqdır. Clustering məlumat analizi, müştəri segmentasiyası, recommender systems (təvsiyə sistemləri), axtarış mühərrikləri, image segmentation (şəkil segmentasiyası), semi-supervised learning, dimensionality reduction və daha çoxu üçün əla bir vasitədir.

Anomaly detection (həmçinin outlier detection adlanır)

Məqsəd "normal" məlumatın necə göründüyünü öyrənmək və bundan istifadə edərək anormal instansları aşkar etməkdir. Bu instanslar *anomalies* və ya *outliers* (kənar dəyərlər), normal instanslar isə *inliers* adlanır. Anomaly detection saxtakarlığın aşkarlanması, istehsalatda qüsurlu məhsulların müəyyən edilməsi, zaman seriyalarında yeni trendlərin müəyyənləşdirilməsi və ya başqa bir modeli öyrətməzdən əvvəl dataset-dən outliers-lərin çıxarılması kimi geniş tətbiq sahələrində faydalıdır (bu, nəticədə yaranan modelin performansını əhəmiyyətli dərəcədə yaxşılaşdırır bilər).

Density estimation

Bu, dataset-i yaradan təsadüfi prosesin *probability density function* (PDF - ehtimal sıxlığı funksiyası) funksiyasını qiymətləndirmək tapşırığıdır.

Density estimation adətən anomaly detection üçün istifadə olunur: çox aşağı sıxlıqlı bölgələrdə yerləşən instanslar çox güman ki, anomalies-dir. O, həmçinin məlumat analizi və vizuallaşdırma üçün də faydalıdır.

k-means-ə keçid. Bir az tort hazır? Biz iki clustering algoritmi – k-means və DBSCAN ilə başlayacağıq, sonra Gaussian mixture modellərini müzakirə edəcəyik və onların density estimation, clustering və anomaly detection üçün necə istifadə oluna biləcəyinə baxacağıq.

Clustering Alqoritmləri: k-means və DBSCAN

Dağlarda gəzinti edərkən, daha əvvəl heç görmədiyiniz bir bitkiyə rast gəlirsiniz. Ətrafa baxırsınız və daha bir neçəsini görürsünüz. Onlar tamamilə eyni deyillər, lakin eyni növə (və ya heç olmasa eyni cinsə) aid olduqlarını bilmək üçün kifayət qədər oxşardırlar. Onların hansı növ olduğunu söyləmək üçün bir botanikə ehtiyacınız ola bilər, lakin oxşar görünən obyektlərin qruplarını müəyyən etmək üçün mütləq ekspertə ehtiyacınız yoxdur. Buna *clustering* deyilir: bu, oxşar instansları müəyyən etmək və onları *clusters*-a, yəni oxşar instanslar qruplarına təyin etmək tapşırığıdır.

Eynilə classification-da olduğu kimi, hər bir instans bir qrupa təyin edilir. Lakin classification-dan fərqli olaraq, clustering Unsupervised bir tapşırıqdır. Şəkil 9-1-in sol tərəfində iris dataset-i (4-cü fəsildə təqdim edilmişdir) təsvir edilmişdir, burada hər bir instansın növü (yəni onun class-ı) fərqli marker ilə göstərilmişdir. Bu, etiketlenmiş bir datasetdir və logistic regression, SVMs və ya random forest classifiers kimi classification alqoritmləri üçün çox uyğundur. Sağ tərəfdə isə eyni dataset var, lakin etikətlər olmadan, buna görə də artıq classification alqoritmindən istifadə edə bilməzsiniz. Burada clustering alqoritmləri

səhnəyə çıxır: onların çoxu aşağı-sol cluster-i asanlıqla aşkar edə bilər. Bizim öz gözümlə də bunu görmək olduqca asandır, lakin yuxarı-sağ cluster-in iki ayrı subcluster-dan ibarət olduğunu görmək o qədər də aydın deyil. Bununla belə, dataset-in burada təsvir olunmayan iki əlavə xüsusiyyəti (sepal uzunluğu və eni) var və clustering alqoritmləri bütün xüsusiyyətlərdən yaxşı istifadə edə bilər, buna görə də əslində onlar üç cluster-i kifayət qədər yaxşı müəyyən edirlər (məsələn, Gaussian mixture modelindən istifadə edərək, 150 instansdan yalnız 5-i səhv cluster-a təyin edilir).

Şəkil 9-1. Classification (sol) və clustering (sağ)

Clustering geniş tətbiq sahələrində istifadə olunur, o cümlədən:

Customer segmentation (Müştəri segmentasiyası)

Müştərilərinizi onların alışları və saytınızdakı fəaliyyətləri əsasında clustering edə bilərsiniz. Bu, müştərilərinizin kim olduğunu və nəyə ehtiyac duyduğunu anlamaq üçün faydalıdır ki, məhsullarınızı və marketing kampaniyalarınızı hər bir segmentə uyğunlaşdırasınız. Məsələn, customer segmentation *recommender systems*-də eyni cluster-dəki digər istifadəçilərin bəyəndiyi məzmunu təklif etmək üçün faydalı ola bilər.

Data analysis (Məlumat analizi)

Yeni bir dataset-i təhlil edərkən, clustering alqoritmini işə salmaq və sonra hər bir cluster-i ayrıca təhlil etmək faydalı ola bilər.

Dimensionality reduction (Ölçülülüyün azaldılması)

Dataset clustering edildikdən sonra, adətən hər bir instansın hər bir cluster ilə *affinity* (yaxınlıq) dərəcəsini ölçmək mümkündür; affinity instansın cluster-a nə dərəcədə uyğun olduğunun hər hansı bir ölçüsüdür. Hər bir instansın xüsusiyyət vektoru x daha sonra onun cluster affinity-lərinin vektoru ilə əvəz edilə bilər. Əgər k cluster varsa, bu vektor k -ölçülü olur. Yeni vektor adətən orijinal xüsusiyyət vektorundan çox daha aşağı ölçülü olur, lakin sonrakı emal üçün kifayət qədər məlumatı qoruya bilər.

Feature engineering (Xüsusiyyət mühəndisliyi)

Cluster affinity-ləri çox vaxt əlavə xüsusiyyətlər kimi faydalı ola bilər. Məsələn, biz Fəsil 2-də California mənzil dataset-inə coğrafi cluster affinity xüsusiyyətləri əlavə etmək üçün k-means istifadə etdik və bu, bizə daha yaxşı performans əldə etməyə kömək etdi.

Anomaly detection (həmçinin outlier detection adlanır)

Bütün cluster-lara qarşı aşağı affinity-yə malik olan hər hansı bir instans çox güman ki, anomaly-dir. Məsələn, saytınızın istifadəçilərini davranışlarına görə clustering etmisinizsə, qeyri-adi davranışı olan istifadəçiləri, məsələn, saniyədə qeyri-adi sayda sorğu göndərənləri aşkar edə bilərsiniz.

Semi-supervised learning

Əgər yalnız bir neçə etiketiniz varsa, clustering həyata keçirə və etikətləri eyni cluster-dəki bütün instanslara yayı bilərsiniz. Bu texnika sonrakı Supervised Learning alqoritmi üçün mövcud etikətlərin sayını əhəmiyyətli dərəcədə artırır və beləliklə onun performansını yaxşılaşdırır.

Search engines (Axtarış mühərrikləri)

Bəzi axtarış mühərrikləri istinad şəklinə oxşar şəkilləri axtarmağa imkan verir. Belə bir sistem qurmaq üçün əvvəlcə verilənlər bazanızdakı bütün şəkillərə clustering alqoritmini tətbiq edərdiniz; oxşar şəkillər eyni cluster-da nəticələnərdi. Sonra istifadəçi istinad şəkli təqdim etdikdə, sizə lazım olan tək şey öyrədilmiş clustering modelindən istifadə edərək bu şəklin cluster-nı tapmaq və sonra sadəcə bu cluster-dan olan bütün şəkilləri qaytarmaqdır.

Image segmentation (Şəkil segmentasiyası)

Pikselləri rənglərinə görə clustering edərək, sonra hər bir pikselin rəngini onun cluster-nın ortalama rəngi ilə əvəz etməklə, bir şəkildəki fərqli rənglərin sayını əhəmiyyətli dərəcədə azaltmaq mümkündür. Image segmentation bir çox object detection və tracking sistemlərində istifadə olunur, çünki bu, hər bir obyektin konturunu aşkar etməyi asanlaşdırır.

Cluster-ın nə olduğuna dair universal bir tərif yoxdur: bu, həqiqətən də kontekstdən asılıdır və müxtəlif alqoritmlər müxtəlif növ cluster-ları tutacaqdır. Bəzi alqoritmlər müəyyən bir nöqtə ətrafında mərkəzləşmiş instansları axtarır, buna *centroid* deyilir. Digərləri sıx yığılmış instansların davamlı bölgələrini axtarır: bu cluster-lar istənilən formanı ala bilər. Bəzi alqoritmlər hierarchical (iyerarxik) olub, cluster-ların cluster-larını axtarır. Və siyahı davam edir.

Bu bölmədə biz iki məşhur clustering alqoritmə – k-means və DBSCAN-a baxacaq və onların bəzi tətbiqlərini, məsələn, nonlinear dimensionality reduction, semi-supervised learning və anomaly detection kimi sahələri araşdıracağıq.

k-means

Şəkil 9-2-də təsvir olunmuş etiketsiz dataset-i nəzərdən keçirin: siz aydın şəkildə beş instans topasını (blob) görə bilərsiniz. *k*-means alqoritmi bu tip dataset-i çox tez və səmərəli şəkildə, çox vaxt cəmi bir neçə iterasiyada clustering edə bilən sadə bir alqoritmdir. Bu, 1957-ci ildə Bell Labs-da Stuart Lloyd tərəfindən pulse-code modulation texnikası kimi təklif edilmiş, lakin şirkətdən kənarda yalnız 1982-ci ildə nəşr edilmişdir. 1965-ci ildə Edward W. Forgy demək olar ki, eyni alqoritmı nəşr etdirmişdir, buna görə də *k*-means bəzən Lloyd–Forgy alqoritmi kimi də adlandırılır.

Şəkil 9-2. Beş instans topasından ibarət etiketsiz dataset

Gəlin bu dataset üzərində bir *k*-means clusterer öyrədək. O, hər bir topanın mərkəzini tapmağa və hər bir instansı ən yaxın topaya təyin etməyə çalışacaq:

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y = make_blobs([...]) # topaları yaradın: y cluster
ID-lərini ehtiva edir, lakin biz onlardan istifadə etməyəcəyik;
bu, proqnozlaşdırmaq istədiyimiz şeydir
k = 5
kmeans = KMeans(n_clusters=k, random_state=42)

y_pred = kmeans.fit_predict(X)
```

Qeyd edək ki, alqoritmin tapmalı olduğu cluster-ların sayı *k*-nı siz təyin etməlisiniz. Bu nümunədə, məlumatlara baxdıqda *k*-nın 5-ə təyin edilməsi olduqca aydındır, lakin ümumiyyətlə bu o qədər də asan deyil. Bunu qısa müddətdə müzakirə edəcəyik.

Hər bir instans beş cluster-dan birinə təyin ediləcəkdir. Clustering kontekstində instansın *label-i* (etiket) algoritmin bu instansa təyin etdiyi cluster-ın indeksidir; bu, classification-da hədəf kimi istifadə olunan class etiketləri ilə qarışdırılmamalıdır (unutmayın ki, clustering Unsupervised Learning tapşırığıdır). KMeans instansı öyrədildiyi instansların proqnozlaşdırılmış etiketlərini `labels_` instans dəyişəni vasitəsilə saxlayır:

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
>>> y_pred is kmeans.labels_
```

```
True
```

Biz həmçinin algoritmin tapdığı beş centroid-ə də baxa bilərik:

```
>>> kmeans.cluster_centers_
array([[ -2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [ -2.80037642,  1.30082566]])
```

Yeni instansları centroid-i ən yaxın olan cluster-a asanlıqla təyin edə bilərsiniz:

```
>>> import numpy as np
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
>>> kmeans.predict(X_new)

array([1, 1, 2, 2], dtype=int32)
```


Cluster-ın qərar sərhədlərini (decision boundaries) çəksəniz, Voronoi tessellation əldə edərsiniz: Şəkil 9-3-ə baxın, burada hər bir centroid X işarəsi ilə təsvir edilmişdir.

Şəkil 9-3. *k-means* qərar sərhədləri (Voronoi tessellation)

İnstanların böyük əksəriyyəti aydın şəkildə müvafiq cluster-a təyin edilmişdir, lakin bir neçə instans, xüsusən də yuxarı-sol cluster ilə mərkəzi cluster arasındakı sərhədə yaxın olanlar, çox güman ki, səhv etiketlenmişdir. Həqiqətən də, *k-means* alqoritmi topaların çox fərqli diametrləri olduqda o qədər də yaxşı işləmir, çünki instansı cluster-a təyin edərkən onun yalnız centroid-ə olan məsafəsinə əhəmiyyət verir.

Hər bir instansı tək bir cluster-a təyin etmək əvəzinə (buna *hard clustering* deyilir), hər bir instansa cluster başına bir bal vermək faydalı ola bilər ki, bu da *soft clustering* adlanır. Bal instans ilə centroid arasındakı məsafə və ya bir similarity score (yaxınlıq balı), məsələn, Fəsil 2-də istifadə etdiyimiz Gaussian radial basis function ola bilər. KMeans sinfində `transform()` metodu hər bir instansdan hər bir centroid-ə qədər olan məsafəni ölçür:

```
>>> kmeans.transform(X_new).round(2)
array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])
```

Bu nümunədə, `X_new`-dakı ilk instans birinci centroid-dən təxminən 2.81, ikinci centroid-dən 0.33, üçüncü centroid-dən 2.90, dördüncü centroid-dən 1.49 və beşinci centroid-dən 2.89 məsafədə yerləşir.

Əgər yüksək ölçülü bir dataset-iniz varsa və onu bu şəkildə transform etsəniz, nəticədə k -ölçülü bir dataset əldə edəcəksiniz: bu transformasiya çox səmərəli nonlinear dimensionality reduction texnikası ola bilər. Alternativ olaraq, Fəsil 2-də olduğu kimi, bu məsafələri başqa bir modeli öyrətmək üçün əlavə xüsusiyyətlər kimi istifadə edə bilərsiniz.

k-means alqoritm

Bəs alqoritm necə işləyir? Tutaq ki, sizə centroid-lər verilib. Siz dataset-dəki bütün instansları onları ən yaxın centroid-ə malik olan cluster-a təyin etməklə asanlıqla etiketləyə bilərsiniz. Əksinə, əgər sizə bütün instans etiketləri verilsəydi, hər bir cluster-ın centroid-ni həmin cluster-dakı instansların ortalamasını hesablamaqla asanlıqla tapa bilərdiniz. Amma sizə nə etiketlər, nə də centroid-lər verilmir, bəs necə davam edə bilərsiniz? Centroid-ləri təsadüfi yerləşdirməklə başlayın (məsələn, dataset-dən k instansı təsadüfi seçib onların yerlərini centroid kimi istifadə etməklə). Sonra instansları etiketləyin, centroid-ləri yeniləyin, instansları etiketləyin, centroid-ləri yeniləyin və s. centroid-lər hərəkət etməyi dayandırana qədər davam edin. Alqoritm məhdud sayda addımda (adətən olduqca az) yığılmasına (convergence) zəmanət verilir. Bunun səbəbi, instanslar və onların ən yaxın centroid-ləri arasındakı ortalama kvadrat məsafənin hər addımda yalnız azala bilməsi və mənfi ola bilmədiyi üçün yığılmanın təmin edilməsidir.

Alqoritmin hərəkətini Şəkil 9-4-də görə bilərsiniz: centroid-lər təsadüfi olaraq başlanğıc vəziyyətinə gətirilir (yuxarı sol), sonra instanslar etiketlenir (yuxarı sağ), sonra centroid-lər yenilənir (mərkəz sol), instanslar yenidən etiketlenir (mərkəz sağ) və s. Gördüyünüz kimi, cəmi

üç iterasiyada alqoritm optimala yaxın görünən bir clustering əldə etmişdir.

QEYD

Alqoritmın hesablama mürəkkəbliyi, ümumiyyətlə, instansların sayı m , cluster-ların sayı k və ölçülərin sayı n -ə görə xətti (linear) olur. Lakin bu, yalnız məlumatlar clustering strukturuna malik olduqda doğrudur. Əgər belə deyilsə, ən pis halda mürəkkəblik instansların sayı ilə eksponensial olaraq arta bilər. Praktikada bu nadir hallarda baş verir və k-means ümumiyyətlə ən sürətli clustering alqoritmlərindən biridir.

Şəkil 9-4. k-means alqoritmi

Alqoritmın yığılmasına zəmanət verilsə də, o, düzgün həllə yığılmaya bilər (yəni, lokal optimuma yığıla bilər): bunun baş verib-verməməsi centroid-in ilkin (initialization) vəziyyətindən asılıdır. Şəkil 9-5-də təsadüfi ilkinləşdirmə addımında şansınız gətirməsə, alqoritmın yığıla biləcəyi iki suboptimal həll göstərilmişdir.

Şəkil 9-5. Uğursuz centroid ilkinləşdirmələri səbəbindən suboptimal həllər

Centroid ilkinləşdirməsini yaxşılaşdırmaqla bu riski azaltmaq üçün bir neçə üsula nəzər salaq.

Centroid ilkinləşdirmə üsulları

Əgər centroid-lərin təxminən harada olması lazım olduğunu bilirsinizsə (məsələn, əgər daha əvvəl başqa bir clustering alqoritmını işə salmışınızsa), `init` hiperparametrini centroid-lərin siyahısını ehtiva edən NumPy massivinə təyin edə və `n_init`-i 1-ə təyin edə bilərsiniz:

```
good_init = np.array([[ -3, 3], [ -3, 2], [ -3, 1], [ -1, 2], [0, 2]])  
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)
```

```
kmeans.fit(X)
```

Başqa bir həll, alqoritmi müxtəlif təsadüfi ilkinləşdirmələrlə bir neçə dəfə işə salmaq və ən yaxşı həlli saxlamaqdır. Təsadüfi ilkinləşdirmələrin sayı `n_init` hiperparametri ilə idarə olunur: standart olaraq bu, 10-a bərabərdir, yəni `fit()` çağırıldıqda yuxarıda təsvir edilən bütün alqoritm 10 dəfə işləyir və Scikit-Learn ən yaxşı həlli saxlayır. Bəs o, ən yaxşı həllin hansı olduğunu necə bilir? O, performans metrikasından istifadə edir! Bu metrika modelin *inertia*-sı (ətaləti) adlanır və instanslar ilə onların ən yaxın centroid-ləri arasındakı kvadrat məsafələrin cəmidir. Şəkil 9-5-də soldakı model üçün təxminən 219.4, sağdakı model üçün 258.6, Şəkil 9-3-dəki model üçün isə cəmi 211.6-dır. KMeans sinfi alqoritmi `n_init` dəfə işlədir və ən aşağı inertia-ya malik modeli saxlayır. Bu nümunədə, Şəkil 9-3-dəki model seçiləcəkdir (əgər çox uğursuz olub, ardıcıl `n_init` təsadüfi ilkinləşdirmə ilə şansımız gətirməsə). Əgər maraqlanırsınızsa, modelin inertia-sı `inertia_` instans dəyişəni vasitəsilə əldə edilə bilər:

```
>>> kmeans.inertia_
```

```
211.59853725816836
```

`score()` metodu mənfəi inertia-nı qaytarır (çünki bir predictor-un

`score()` metodu hər zaman Scikit-Learn-in "böyük daha yaxşıdır"

qaydasına əməl etməlidir: əgər bir predictor digərindən daha yaxşıdırsa,

onun `score()` metodu daha yüksək bal qaytarmalıdır):

```
>>> kmeans.score(X)
```

```
-211.5985372581684
```

k-means alqoritminə mühüm bir təkmilləşdirmə olan *k-means++* 2006-cı ildə David Arthur və Sergei Vassilvitskii tərəfindən təklif edilmişdir. Onlar bir-birindən uzaq olan centroid-ləri seçməyə meyli olan daha ağıllı bir ilkinləşdirmə addımını təqdim etdilər və bu təkmilləşdirmə *k*-means alqoritminin suboptimal həllə yığılma ehtimalını xeyli azaldır. Məqalə göstərdi ki, daha ağıllı ilkinləşdirmə addımı üçün tələb olunan əlavə hesablama, optimal həlli tapmaq üçün alqoritmin işə salınma sayını kəskin şəkildə azaltmağa imkan verdiyi üçün buna dəyər. *k-means++* ilkinləşdirmə alqoritmi belə işləyir:

1. Bir centroid $c(1)$ götürün, dataset-dən bərabər təsadüfi olaraq seçilir.
2. Yeni bir centroid $c(i)$ götürün, $x(i)$ instansını ehtimalla seçin: $Dx(i)^2 / \sum_{j=1}^m Dx(j)^2$, burada $D(\cdot)$ instans $x(i)$ ilə artıq seçilmiş ən yaxın centroid arasındakı məsafədir. Bu ehtimal paylanması, artıq seçilmiş centroid-lərdən daha uzaqda olan instansların centroid kimi seçilmə ehtimalının daha yüksək olmasını təmin edir.
3. Bütün k centroid seçilənə qədər əvvəlki addımı təkrarlayın.

KMeans sinfi standart olaraq bu ilkinləşdirmə metodundan istifadə edir.

Accelerated *k*-means və mini-batch *k*-means

k-means alqoritminə digər bir təkmilləşdirmə 2003-cü ildə Charles Elkan tərəfindən təklif edilmişdir. Çoxlu cluster-ları olan böyük dataset-lərdə, alqoritm bir çox lazımsız məsafə hesablamalarından qaçmaqla sürətləndirilə bilər. Elkan buna üçbucaq bərabərsizliyindən (yəni, iki nöqtə arasındakı ən qısa məsafənin həmişə düz xətt olduğundan) istifadə edərək və instanslar ilə centroid-lər arasındakı məsafələr üçün aşağı və yuxarı hədləri izləməklə nail olmuşdur. Lakin Elkan-ın alqoritmi

həmişə təlimi sürətləndirmir və bəzən hətta onu əhəmiyyətli dərəcədə yavaşlata bilər; bu, dataset-dən asılıdır. Buna baxmayaraq, əgər sınaq istəyirsinizsə, `algorithm="elkan"` təyin edin.

k-means alqoritminin daha bir mühüm variantı 2010-cu ildə David Sculley tərəfindən təklif edilmişdir. Hər bir iterasiyada tam dataset əvəzinə, alqoritm mini-batch-lərdən istifadə etməyə qadirdir və centroid-ləri hər iterasiyada yalnız bir az hərəkət etdirir. Bu, alqoritm sürətləndirir (adətən üç-dörd dəfə) və yaddaşa sığmayan nəhəng dataset-ləri clustering etməyə imkan verir. Scikit-Learn bu alqoritm MiniBatchKMeans sinfində tətbiq edir və siz onu KMeans sinfi kimi istifadə edə bilərsiniz:

```
from sklearn.cluster import MiniBatchKMeans
```

```
minibatch_kmeans = MiniBatchKMeans(n_clusters=5,  
random_state=42)
```

```
minibatch_kmeans.fit(X)
```

Əgər dataset yaddaşa sığmırsa, ən sadə seçim Fəsil 8-də incremental PCA üçün etdiyimiz kimi `memmap` sinfindən istifadə etməkdir. Alternativ olaraq, hər dəfə bir mini-batch-i `partial_fit()` metoduna ötürə bilərsiniz, lakin bu, çox daha çox iş tələb edəcək, çünki çoxsaylı ilkinləşdirmələr aparmalı və ən yaxşısını özünüz seçməlisiniz.

Mini-batch *k*-means alqoritmı adi *k*-means alqoritmindən çox daha sürətli olsa da, onun inertia-sı ümumiyyətlə bir qədər pisdır. Bunu Şəkil 9-6-da görə bilərsiniz: soldakı qrafik müxtəlif cluster sayları *k* üçün əvvəlki beş-topa dataset-də öyrədilmiş mini-batch və adi *k*-means modellərinin inertia-larını müqayisə edir. İki əyri arasındakı fərq kiçikdir,

lakin görünəndir. Sağdakı qrafikdə isə görə bilərsiniz ki, mini-batch k-means bu dataset-də adi k-means-dan təxminən 3.5 dəfə sürətlidir.

Şəkil 9-6. Mini-batch k-means k-means-dan daha yüksək inertia-ya malikdir (sol), lakin daha sürətlidir (sağ), xüsusən k artdıqca

Optimal cluster sayının tapılması

İndiyə qədər biz cluster-ların sayını k -nı 5-ə təyin etdik, çünki məlumatlara baxdıqda bunun düzgün say olduğu aydın idi. Lakin ümumiyyətlə k -nı necə təyin edəcəyinizi bilmək o qədər də asan olmayacaq və əgər onu səhv qiymətə təyin etsəniz, nəticə olduqca pis ola bilər. Şəkil 9-7-də gördüyünüz kimi, bu dataset üçün k -nı 3 və ya 8-ə təyin etmək kifayət qədər pis modellərlə nəticələnir.

Siz düşünə bilərsiniz ki, sadəcə ən aşağı inertia-ya malik modeli seçə bilərsiniz. Təəssüf ki, bu o qədər də sadə deyil. $k=3$ üçün inertia təxminən 653.2-dir ki, bu da $k=5$ (211.6) üçün olandan çox daha yüksəkdir. Lakin $k=8$ ilə inertia cəmi 119.1-dir. k -nı seçməyə çalışarkən inertia yaxşı bir performans metrikası deyil, çünki k -nı artırıdıqca o, daim aşağı düşür. Həqiqətən də, nə qədər çox cluster varsa, hər bir instans öz ən yaxın centroid-nə bir o qədər yaxın olacaq və buna görə də inertia bir o qədər aşağı olacaq. Gəlin inertia-nı k -nın funksiyası kimi çəkək. Bunu etdikdə, əyri çox vaxt *elbow* (dirsək) adlanan bir əyilmə nöqtəsini ehtiva edir (bax Şəkil 9-8).

Şəkil 9-7. Cluster sayı üçün pis seçimlər: k çox kiçik olduqda ayrı-ayrı cluster-lar birləşir (sol), k çox böyük olduqda isə bəzi cluster-lar çoxlu hissələrə bölünür (sağ)

Şəkil 9-8. Inertia-nın cluster sayı k -nın funksiyası kimi qrafiki

Gördüyünüz kimi, k -nı 4-ə qədər artırdıqca inertia çox sürətlə düşür, lakin sonra k -nı artırmağa davam etdikcə daha yavaş azalır. Bu əyri təxminən bir qolun formasına malikdir və $k = 4$ -də bir dirsək var. Beləliklə, əgər daha yaxşısını bilməsəydik, 4-ün yaxşı bir seçim olduğunu düşünə bilərdik: daha aşağı hər hansı bir qiymət dramatik olardı, daha yüksək qiymət isə çox kömək etməzdi və biz sadəcə yaxşı cluster-ları heç bir səbəb olmadan yarıya bölmüş olardıq.

Cluster sayının ən yaxşı dəyərini seçmək üçün bu texnika olduqca kobuddur. Daha dəqiq (lakin eyni zamanda daha çox hesablama tələb edən) yanaşma *silhouette score* (siluet balı) istifadə etməkdir. Bu, bütün instanslar üzərindəki ortalama *silhouette coefficient* (siluet əmsalı) dır. Bir instansın silhouette coefficient-i $(b - a) / \max(a, b)$ bərabərdir, burada a eyni cluster-dakı digər instanslara ortalama məsafədir (yəni, ortalama intra-cluster məsafəsi) və b ən yaxın növbəti cluster-ın instanslarına olan ortalama məsafədir (yəni, instansın öz cluster-i istisna olmaqla, b -ni minimuma endirən cluster kimi təyin olunan ən yaxın cluster-a olan məsafə). Silhouette coefficient -1 ilə +1 arasında dəyişə bilər. +1-ə yaxın bir əmsal o deməkdir ki, instans öz cluster-i daxilində yaxşı yerləşir və digər cluster-lardan uzaqdır; 0-a yaxın bir əmsal onun cluster sərhədinə yaxın olduğunu göstərir; nəhayət, -1-ə yaxın bir əmsal instansın səhv cluster-a təyin edilmiş ola biləcəyini göstərir.

Silhouette score-u hesablamaq üçün, Scikit-Learn-ın

`silhouette_score()` funksiyasından istifadə edə bilərsiniz, ona dataset-dəki bütün instansları və onlara təyin edilmiş etiketləri verərək:

```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
```


0.655517642572828

Gəlin müxtəlif cluster sayları üçün silhouette score-larını müqayisə edək (bax Şəkil 9-9).

Şəkil 9-9. Silhouette score istifadə edərək k cluster sayının seçilməsi

Gördüyünüz kimi, bu vizuallaşdırma əvvəlkindən daha zəngindir: o, $k = 4$ -ün çox yaxşı bir seçim olduğunu təsdiqləsə də, həm də $k = 5$ -in kifayət qədər yaxşı olduğunu və $k = 6$ və ya 7-dən daha yaxşı olduğunu vurğulayır. Bu, inertia-ları müqayisə edərkən görünmürdü.

Daha da informativ bir vizuallaşdırma, hər bir instansın silhouette coefficient-i, onların təyin olunduğu cluster-lara və əmsalın dəyərinə görə sıralanmış şəkildə çəkildikdə əldə edilir. Buna *silhouette diagram* (siluet diaqramı) deyilir (bax Şəkil 9-10). Hər bir diaqram, hər bir cluster üçün bir bıçaq forması ehtiva edir. Formanın hündürlüyü cluster-dakı instansların sayını, eni isə cluster-dakı instansların sıralanmış silhouette coefficient-lərini göstərir (enli daha yaxşıdır).

Şaquli kəsikli xətlər hər bir cluster sayı üçün ortalama silhouette score-u təmsil edir. Bir cluster-dakı instansların əksəriyyəti bu score-dan daha aşağı bir əmsala malik olduqda (yəni, bir çox instans kəsikli xəttədən əvvəl dayanıb, onun solunda bitərsə), o cluster olduqca pisdır, çünki bu o deməkdir ki, onun instansları digər cluster-lara çox yaxındır. Burada görə bilərik ki, $k = 3$ və ya 6 olduqda pis cluster-lar alırıq. Lakin $k = 4$ və ya 5 olduqda cluster-lar olduqca yaxşı görünür: instansların əksəriyyəti kəsikli xəttədən kənara, sağa və 1.0-a yaxın uzanır.

$k = 4$ olduqda, 1-ci indeksli cluster (aşağıdan ikinci) olduqca böyükdür. $k = 5$ olduqda, bütün cluster-lar oxşar ölçülərə malikdir. Beləliklə, $k = 4$

üçün ümumi silhouette score $k = 5$ üçün olandan bir qədər böyük olsa da, oxşar ölçülü cluster-lar əldə etmək üçün $k = 5$ istifadə etmək yaxşı bir fikir kimi görünür.

Şəkil 9-10. Müxtəlif k qiymətləri üçün silhouette diaqramlarının təhlili

k-means-in məhdudiyyətləri

Çoxsaylı üstünlüklərinə, xüsusən də sürətli və miqyaslanı bilən olmasına baxmayaraq, k-means mükəmməl deyil. Gördüyümüz kimi, suboptimal həllərdən qaçmaq üçün alqoritmi bir neçə dəfə işə salmaq lazımdır, əlavə olaraq cluster-ların sayını təyin etməlisiniz ki, bu da olduqca əngəlli ola bilər. Üstəlik, k-means cluster-lar müxtəlif ölçülərə, müxtəlif sıxlıqlara və ya qeyri-sferik formalara malik olduqda o qədər də yaxşı davranmır. Məsələn, Şəkil 9-11 k-means-ın müxtəlif ölçülərə, sıxlıqlara və istiqamətlərə malik üç ellipsoidal cluster-dən ibarət bir dataset-i necə clustering etdiyini göstərir.

Gördüyünüz kimi, bu həllərin heç biri yaxşı deyil. Soldakı həll daha yaxşıdır, lakin o, hələ də orta cluster-in 25%-ni kəsib sağdakı cluster-a təyin edir. Sağdakı həll isə sadəcə dəhşətlidir, baxmayaraq ki, onun inertia-sı daha aşağıdır. Beləliklə, məlumatlardan asılı olaraq, müxtəlif clustering alqoritmləri daha yaxşı performans göstərə bilər. Bu tip elliptik cluster-larda Gaussian mixture modelləri əla işləyir.

Şəkil 9-11. k-means bu ellipsoidal topaları düzgün clustering edə bilmir

İPUCU

k-means işə salmazdan əvvəl giriş xüsusiyyətlərini miqyaslandırmaq (bax Fəsil 2) vacibdir, əks halda cluster-lar çox uzanmış ola bilər və

k-means zəif performans göstərəcək. Xüsusiyyətlərin miqyaslandırılması bütün cluster-ların gözəl və sferik olacağına zəmanət vermir, lakin ümumiyyətlə k-means-a kömək edir.

İndi clustering-dən faydalana biləcəyimiz bir neçə üsula baxaq. Biz k-means istifadə edəcəyik, lakin digər clustering alqoritmləri ilə də sınaqdan keçirməkdən çekinmeyin.

Clustering-in Image Segmentation üçün istifadəsi

Image segmentation (şəkil seqmentasiyası) bir şəkli çoxsaylı seqmentlərə ayırmaq tapşırığıdır. Bunun bir neçə variantı var:

- *Color segmentation*-də (rəng seqmentasiyası) oxşar rəngə malik piksellər eyni seqmentə təyin edilir. Bu, bir çox tətbiqlərdə kifayətdir. Məsələn, bir bölgədə ümumi meşə sahəsinin nə qədər olduğunu ölçmək üçün peyk şəkillərini təhlil etmək istəyirsinizsə, color segmentation kifayət edə bilər.
- *Semantic segmentation*-də (semantik seqmentasiya) eyni obyekt növünün bir hissəsi olan bütün piksellər eyni seqmentə təyin edilir. Məsələn, özü idarə edən avtomobilin görmə sistemində piyada şəklinin bir hissəsi olan bütün piksellər "piyada" seqmentinə təyin edilə bilər (bütün piyadaları ehtiva edən bir seqment olardı).
- *Instance segmentation*-də (instans seqmentasiyası) eyni fərdi obyektin bir hissəsi olan bütün piksellər eyni seqmentə təyin edilir. Bu halda hər bir piyada üçün fərqli bir seqment olardı.

Bu gün semantic və ya instance segmentation sahəsində ən müasir (state-of-the-art) üsullar convolutional neural networks (bax Fəsil 14) əsasında mürəkkəb arxitekturalardan istifadə etməklə əldə edilir. Bu fəsildə biz daha sadə olan *color segmentation* tapşırığına diqqət yetirəcəyik, k-means istifadə edərək.

Biz Pillow paketini (Python Imaging Library, PIL-in davamçısı) idxal etməklə başlayacağıq, sonra ondan *ladybug.png* şəklini yükləmək üçün

istifadə edəcəyik (Şəkil 9-12-nin yuxarı-sol şəklinə baxın), fərz edək ki, o, filepath ünvanında yerləşir:

```
>>> import PIL
>>> image = np.asarray(PIL.Image.open(filepath))
>>> image.shape
```

```
(533, 800, 3)
```

Şəkil 3D massiv kimi təmsil olunur. Birinci ölçünün ölçüsü hündürlük; ikincisi en; üçüncüsü isə rəng kanallarının sayıdır, bu halda qırmızı, yaşıl və mavi (RGB). Başqa sözlə, hər bir piksel üçün 0 ilə 255 arasında işarəsiz 8-bit tam ədədlər kimi qırmızı, yaşıl və mavi intensivliklərini ehtiva edən 3D vektor var. Bəzi şəkillər daha az kanala malik ola bilər (məsələn, yalnız bir kanalı olan bozqanadlı şəkillər), bəzi şəkillər isə daha çox kanala malik ola bilər (məsələn, şəffaflıq üçün əlavə *alpha channel* olan şəkillər və ya tez-tez əlavə işıq tezlikləri üçün kanallar ehtiva edən peyk şəkilləri).

Aşağıdakı kod massivi uzun bir RGB rəng siyahısı əldə etmək üçün yenidən formalaşdırır (reshape), sonra bu rəngləri k-means istifadə edərək səkkiz cluster ilə clustering edir. O, hər bir piksel üçün ən yaxın cluster mərkəzini (yəni, hər pikselin cluster-nın ortalama rəngini) ehtiva edən `segmented_img` massivi yaradır və nəhayət bu massivi orijinal şəkil formasına qaytarır. Üçüncü sətir qabaqcıl NumPy indeksləşdirməsindən istifadə edir; məsələn, `kmeans_.labels_`-dəki ilk 10 etiket 1-ə bərabərdirsə, `segmented_img`-dəki ilk 10 rəng `kmeans.cluster_centers_[1]`-ə bərabərdir:

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8, random_state=42).fit(X)
```

```
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
```

```
segmented_img = segmented_img.reshape(image.shape)
```

Bu, Şəkil 9-12-nin yuxarı sağında göstərilən şəkli çıxarır. Siz şəkildə göstərildiyi kimi müxtəlif sayda cluster-larla sınaq keçirə bilərsiniz. Səkkizdən az cluster istifadə etdikdə, diqqət yetirin ki, ladybug-un parlaq qırmızı rəngi özünə aid bir cluster əldə edə bilmir: o, ətraf mühitin rəngləri ilə birləşir. Bunun səbəbi, k-means-ın oxşar ölçülü cluster-lara üstünlük verməsidir. Ladybug kiçikdir – şəklin qalan hissəsindən çox kiçikdir – buna görə də rəngi parlaq olsa da, k-means ona bir cluster ayıra bilmir.

Şəkil 9-12. Müxtəlif sayda rəng cluster-ları ilə k-means istifadə edərək image segmentation

Bu o qədər də çətin deyildi, elə deyilmi? İndi clustering-in başqa bir tətbiqinə baxaq.

Clustering-in Semi-Supervised Learning üçün istifadəsi

Clustering üçün başqa bir istifadə halı semi-supervised learning-dədir, o zaman çoxlu sayda etiketsiz instans və çox az sayda etiketli instansımız olur. Bu bölmədə, biz digits dataset-indən istifadə edəcəyik. Bu, 0-dan 9-a qədər rəqəmləri təmsil edən 1,797 bozqanadlı 8×8 şəkildən ibarət sadə MNIST-ə bənzər bir datasetdir. Əvvəlcə dataset-i yükləyək və bölək (o, artıq qarışdırılmışdır):

```
from sklearn.datasets import load_digits
```

```
X_digits, y_digits = load_digits(return_X_y=True)
```

```
X_train, y_train = X_digits[:1400], y_digits[:1400]
```

```
X_test, y_test = X_digits[1400:], y_digits[1400:]
```

Biz yalnız 50 instans üçün etiketlərimiz olduğunu fərz edəcəyik. Baza performansını əldə etmək üçün gəlin bu 50 etiketli instans üzərində bir logistic regression modeli öyrədək:

```
from sklearn.linear_model import LogisticRegression
```

```
n_labeled = 50
```

```
log_reg = LogisticRegression(max_iter=10_000)
```

```
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

Sonra modelin test seti üzərindəki dəqiqliyini (accuracy) ölçə bilərik (qeyd edək ki, test seti etiketli olmalıdır):

```
>>> log_reg.score(X_test, y_test)
```

```
0.7481108312342569
```

Modelin dəqiqliyi cəmi 74.8%-dir. Bu, o qədər də yaxşı deyil: həqiqətən də, əgər model tam training seti üzərində öyrətməyə çalışsanız, onun təxminən 90.7% dəqiqliyə çatacağını görərsiniz. Gəlin görək necə daha yaxşı edə bilərik. Əvvəlcə, training setini 50 cluster-a bölək. Sonra, hər bir cluster üçün, centroid-ə ən yaxın olan şəkli tapaq. Bu şəkilləri *representative images* (nümayəndə şəkillər) adlandıracağıq:

```
k = 50
```

```
kmeans = KMeans(n_clusters=k, random_state=42)
```

```
X_digits_dist = kmeans.fit_transform(X_train)
```

```
representative_digit_idx = np.argmin(X_digits_dist, axis=0)
```

```
X_representative_digits =
```

```
X_train[representative_digit_idx]
```

Şəkil 9-13 50 nümayəndə şəklini göstərir.

Şəkil 9-13. Əlli nümayəndə rəqəm şəkli (hər cluster üçün bir)

Gəlin hər bir şəklə baxaq və onları əl ilə etiketləyək:

```
y_representative_digits = np.array([1, 3, 6, 0, 7, 9,
2, ..., 5, 1, 9, 9, 3, 7])
```

İndi bizdə cəmi 50 etiketli instans olan bir dataset var, lakin təsadüfi instanslar əvəzinə, onların hər biri öz cluster-nın nümayəndə şəklidir.

Gəlin görək performans daha yaxşıdır mı:

```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_representative_digits,
y_representative_digits)
>>> log_reg.score(X_test, y_test)
```

```
0.8488664987405542
```

Vay! Biz dəqiqliyi 74.8%-dən 84.9%-ə qaldırdıq, baxmayaraq ki, modeli hələ də yalnız 50 instans üzərində öyrədirik. İnstansları, xüsusən də bunu əl ilə ekspertlər tərəfindən etmək lazım olduqda, etiketləmək çox vaxt bahalı və ağırlı olduğundan, təsadüfi instanslar əvəzinə nümayəndə instansları etiketləmək yaxşı bir fikirdir.

Lakin bəlkə daha da irəli gedə bilərik: əgər etiketləri eyni cluster-dakı bütün digər instanslara yaysaq nə olar? Buna *label propagation* (etiket yayılması) deyilir:

```
y_train_propagated = np.empty(len(X_train), dtype=np.int64)
for i in range(k):
```

```
y_train_propagated[kmeans.labels_ == i] =  
y_representative_digits[i]
```

İndi modeli yenidən öyrədək və performansına baxaq:

```
>>> log_reg = LogisticRegression()  
>>> log_reg.fit(X_train, y_train_propagated)  
>>> log_reg.score(X_test, y_test)
```

```
0.8942065491183879
```

Biz daha bir əhəmiyyətli dəqiqlik artımı əldə etdik! Gəlin görək, hər bir cluster mərkəzindən ən uzaq olan 1% instansları nəzərə almamaqla daha da yaxşılaşa bilərikmi: bu, bəzi outlier-ləri aradan qaldırmalıdır. Aşağıdakı kod əvvəlcə hər bir instansdan onun ən yaxın cluster mərkəzinə qədər olan məsafəni hesablayır, sonra hər bir cluster üçün ən böyük 1% məsafəni –1-ə təyin edir. Nəhayət, –1 məsafəsi ilə işarələnmiş bu instanslar olmadan bir set yaradır:

```
percentile_closest = 99
```

```
X_cluster_dist = X_digits_dist[np.arange(len(X_train)),  
kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist,  
percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1
```

```
partially_propagated = (X_cluster_dist != -1)
```

```
X_train_partially_propagated = X_train[partially_propagated]
```



```
y_train_partially_propagated =  
y_train_propagated[partially_propagated]
```

İndi modeli bu qismən yayılmış dataset üzərində yenidən öyrədək və hansı dəqiqliyi əldə etdiyimizə baxaq:

```
>>> log_reg = LogisticRegression(max_iter=10_000)  
>>> log_reg.fit(X_train_partially_propagated,  
y_train_partially_propagated)  
>>> log_reg.score(X_test, y_test)
```

```
0.9093198992443325
```

Əla! Cəmi 50 etiketli instansla (orta hesabla hər sinif üçün yalnız 5 nümunə!) biz 90.9% dəqiqlik əldə etdik ki, bu da əslində tam etiketlenmiş digits dataset-də əldə etdiyimiz performansdan (90.7%) bir qədər yüksəkdir. Bu, qismən bəzi outlier-ləri atdığımıza, qismən də yayılmış etiketlərin həqiqətən yaxşı olmasına bağlıdır – onların dəqiqliyi təxminən 97.5%-dir, aşağıdakı kodun göstərdiyi kimi:

```
>>> (y_train_partially_propagated ==  
y_train[partially_propagated]).mean()
```

```
0.9755555555555555
```

İPUCU

Scikit-Learn həmçinin etiketləri avtomatik yaya bilən iki sinif təklif edir:

`sklearn.semi_supervised` paketindəki `LabelSpreading` və

`LabelPropagation`. Hər iki sinif bütün instanslar arasında bir oxşarlıq

matrisi qurur və etiketli instanslardan oxşar etiketsiz instanslara

etiketləri iterativ olaraq yayır. Eyni paketdə çox fərqli bir sinif olan

`SelfTrainingClassifier` da var: siz ona əsas classifier verirsiniz

(məsələn, `RandomForestClassifier`) və o, onu etiketli instanslar üzərində öyrədir, sonra etiketsiz nümunələr üçün etikətləri proqnozlaşdırmaq üçün istifadə edir. Daha sonra o, ən çox əmin olduğu etikətlərlə training setini yeniləyir və daha çox etiket əlavə edə bilməyə qədər bu təlim və etikətləmə prosesini təkrarlayır. Bu texnikalar sehrli güllə deyillər, lakin bəzən modelinizə bir az kömək edə bilərlər.

ACTIVE LEARNING (AKTİV ÖYRƏNMƏ)

Modelinizi və training setinizi yaxşılaşdırmaq üçün növbəti addım, bir neçə dövr *active learning* etmək ola bilər. Bu, insan ekspertinin öyrənmə alqoritmı ilə qarşılıqlı əlaqədə olduğu və alqoritm tələb etdikdə müəyyən instanslar üçün etikətlər təqdim etdiyi bir prosesdir. Active learning üçün çoxlu müxtəlif strategiyalar var, lakin ən çox yayılmışlardan biri *uncertainty sampling* (qeyri-müəyyənlik nümunəsi) adlanır. O, belə işləyir:

1. Model indiyə qədər toplanmış etiketli instanslar üzərində öyrədilir və bu model bütün etiketsiz instanslar üzərində proqnozlar vermək üçün istifadə olunur.
2. Modelin ən az əmin olduğu instanslar (yəni, təxmin edilən ehtimalın ən aşağı olduğu yerlər) ekspertə etikətləmək üçün verilir.
3. Performans artımı etikətləmə sayını dəyərləndirməyə dəyər olmayana qədər bu prosesi təkrarlayırsınız.

Digər active learning strategiyalarına ən böyük model dəyişikliyi və ya modelin validasiya səhvində ən böyük azalma ilə nəticələnəcək instansların etikətlənməsi və ya müxtəlif modellərin (məsələn, SVM və random forest) razılaşmadığı instansların etikətlənməsi daxildir.

Gaussian mixture modellərinə keçməzdən əvvəl, gəlin DBSCAN-a nəzər salaq. Bu, yerli sıxlıq təxmininə əsaslanan çox fərqli bir yanaşmanı

nümayiş etdirən digər məşhur clustering alqoritmidir. Bu yanaşma alqoritmə özbaşına formalı cluster-ları müəyyən etməyə imkan verir.

DBSCAN

Density-based spatial clustering of applications with noise (DBSCAN)

alqoritm cluster-ları yüksək sıxlıqlı davamlı bölgələr kimi müəyyən edir.

O, belə işləyir:

- Hər bir instans üçün alqoritm ondan kiçik bir ϵ (epsilon) məsafəsi daxilində neçə instansın yerləşdiyini sayır. Bu bölgə instansın ϵ -neighborhood (ϵ -qonşuluğu) adlanır.
- Əgər bir instansın ϵ -qonşuluğunda ən azı `min_samples` instans varsa (özü də daxil olmaqla), o, *core instance* (nüvə instansı) hesab edilir. Başqa sözlə, core instance-lar sıx bölgələrdə yerləşənlərdir.
- Bir core instance-ın qonşuluğundakı bütün instanslar eyni cluster-a məxsusdur. Bu qonşuluq digər core instance-ları da ehtiva edə bilər; buna görə də, qonşu core instance-ların uzun bir ardıcılığı tək bir cluster təşkil edir.
- Core instance olmayan və onun qonşuluğunda heç biri olmayan hər hansı bir instans anomaly (anomaliya) hesab edilir.

Bu alqoritm bütün cluster-lar aşağı sıxlıqlı bölgələrlə yaxşı ayrıldıqda yaxşı işləyir. Scikit-Learn-dakı DBSCAN sinfi gözlədiyiniz qədər sadədir.

Gəlin onu 5-ci fəsildə təqdim edilən moons dataset-də sınayaq:

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
```

```
X, y = make_moons(n_samples=1000, noise=0.05)
dbscan = DBSCAN(eps=0.05, min_samples=5)
```

```
dbscan.fit(X)
```

Bütün instansların etiketləri indi `labels_` instans dəyişənində mövcuddur:

```
>>> dbscan.labels_
```

```
array([ 0,  2, -1, -1,  1,  0,  0,  0,  2,  5, [...],  
       3,  3,  4,  2,  6,  3])
```

Diqqət yetirin ki, bəzi instansların cluster indeksi -1 -ə bərabərdir, bu o deməkdir ki, onlar alqoritm tərəfindən anomaly kimi qəbul edilir. Core instance-ların indeksləri `core_sample_indices_` instans dəyişəninə, core instance-ların özləri isə `components_` instans dəyişəninə mövcuddur:

```
>>> dbscan.core_sample_indices_
```

```
array([ 0,  4,  5,  6,  7,  8, 10, 11, [...], 993,  
       995, 997, 998, 999])
```

```
>>> dbscan.components_
```

```
array([[ -0.02137124,  0.40618608],  
       [ -0.84192557,  0.53058695],  
       [...],  
       [ 0.79419406,  0.60777171]])
```

Bu clustering Şəkil 9-14-ün sol tərəfindəki qrafikdə təsvir edilmişdir.

Gördüyünüz kimi, o, olduqca çox anomaly, üstəlik yeddi müxtəlif cluster müəyyən etdi. Nə qədər məyusedici! Xoşbəxtlikdən, əgər hər bir instansın qonşuluğunu `eps`-i 0.2 -yə qaldıraraq genişləndirsək, sağdakı qrafikdə mükəmməl görünən clustering əldə edirik. Gəlin bu model ilə davam edək.

Şəkil 9-14. İki fərqli qonşuluq radiusu istifadə edən DBSCAN clustering

Təəccüblü olsa da, DBSCAN sinfində `predict()` metodu yoxdur, baxmayaraq ki, `fit_predict()` metodu var. Başqa sözlə, o, yeni bir instansın hansı cluster-a aid olduğunu proqnozlaşdırma bilmir. Bu qərar,

müxtəlif classification alqoritmlərinin müxtəlif tapşırıqlar üçün daha yaxşı ola biləcəyi üçün verilmişdir, buna görə də müəlliflər istifadəçinin hansını istifadə edəcəyini seçməsinə icazə vermək qərarına gəldilər. Üstəlik, onu tətbiq etmək çətin deyil. Məsələn, gəlin bir KNeighborsClassifier öyrədək:

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=50)
```

```
knn.fit(dbscan.components_,  
        dbscan.labels_[dbscan.core_sample_indices_])
```

İndi, bir neçə yeni instans verildikdə, onların çox güman ki, hansı cluster-lara aid olduğunu proqnozlaşdır və hətta hər bir cluster üçün ehtimalı qiymətləndirə bilərik:

```
>>> X_new = np.array([[ -0.5,  0], [ 0,  0.5], [ 1, -0.1], [ 2,  1]])  
>>> knn.predict(X_new)  
array([1, 0, 1, 0])  
>>> knn.predict_proba(X_new)  
array([[0.18, 0.82],  
       [1.   , 0.   ],  
       [0.12, 0.88],  
       [1.   , 0.   ]])
```

Qeyd edək ki, biz classifier-i yalnız core instance-lar üzərində öyrətdik, lakin bütün instanslar və ya anomaly-lar istisna olmaqla hamısı üzərində öyrətməyi də seçə bilərdik: bu seçim son tapşırıqdan asılıdır.

Qərar sərhədi Şəkil 9-15-də təsvir edilmişdir (xaçlar X_new-dəki dörd instansı təmsil edir). Diqqət yetirin ki, training set-də heç bir anomaly olmadığı üçün classifier həmişə bir cluster seçir, hətta o cluster uzaqda

olsa belə. Maksimum məsafə tətbiq etmək olduqca sadədir, bu halda hər iki cluster-dan uzaqda olan iki instans anomaly kimi təsnif ediləcək.

Bunu etmək üçün KNeighborsClassifier-ın `kneighbors()` metodundan istifadə edin. Verilmiş instanslar toplusu üçün o, training set-dəki k -ən yaxın qonşuların məsafələrini və indekslərini qaytarır (hər biri k sütunu olan iki matris):

```
>>> y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
>>> y_pred =
dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
>>> y_pred[y_dist > 0.2] = -1
>>> y_pred.ravel()

array([-1,  0,  1, -1])
```

Şəkil 9-15. İki cluster arasında qərar sərhədi

Qısacası, DBSCAN istənilən sayda və istənilən formalı cluster-ları müəyyən edə bilən çox sadə, lakin güclü bir alqoritmdir. O, outlier-lərə qarşı davamlıdır və cəmi iki hiperparametrə (`eps` və `min_samples`) malikdir. Əgər cluster-lar arasında sıxlıq əhəmiyyətli dərəcədə dəyişirsə və ya bəzi cluster-ların ətrafında kifayət qədər aşağı sıxlıqlı bölgə yoxdursa, DBSCAN bütün cluster-ları düzgün əhatə etməkdə çətinlik çəkə bilər. Üstəlik, onun hesablama mürəkkəbliyi təxminən $O(mn)$ olduğundan, böyük dataset-lərə yaxşı miqyaslanmır.

İPUCU

Siz həmçinin *hierarchical DBSCAN* (HDBSCAN) – `scikit-learn-contrib` layihəsində tətbiq edilmişdir – sınaq etmək istəyə bilərsiniz, çünki o, adətən DBSCAN-dan müxtəlif sıxlıqlı cluster-ları tapmaqda daha yaxşıdır.

Digər Clustering Alqoritmləri

Scikit-Learn sizin baxmalı olduğunuz daha bir neçə clustering alqoritmini tətbiq edir. Onların hamısını burada ətraflı izah edə bilmərəm, lakin qısa bir xülasə təqdim edirəm:

Agglomerative clustering

Cluster-ların iyerarxiyası aşağıdan yuxarıya doğru qurulur. Suyun üzərində üzən və tədricən bir-birinə yapışan çoxlu kiçik qabarcıqlar düşünün, ta ki bir böyük qrup qabarcıq olsun. Eynilə, hər bir iterasiyada agglomerative clustering ən yaxın cüt cluster-i birləşdirir (fərdi instanslardan başlayaraq). Əgər birləşən hər bir cluster cütü üçün bir budaq çəksəniz, yarpaqları fərdi instanslar olan cluster-ların ikili ağacını alardınız. Bu yanaşma müxtəlif formalı cluster-ları tuta bilər; həmçinin müəyyən bir cluster miqyasını seçməyə məcbur etmək əvəzinə çevik və informativ bir cluster ağacı yaradır və istənilən cütlük məsafəsi ilə istifadə edilə bilər. Əgər bir connectivity matrix (qonşuluq matrisi) – hansı cüt instansların qonşu olduğunu göstərən seyrək matris (məsələn, `sklearn.neighbors.kneighbors_graph()` tərəfindən qaytarılan) – təqdim etsəniz, çox sayda instansa yaxşı miqyaslanı bilər. Connectivity matrix olmadan, alqoritm böyük dataset-lərə yaxşı miqyaslanmır.

BIRCH

Balanced iterative reducing and clustering using hierarchies (BIRCH)

alqoritmi xüsusilə çox böyük dataset-lər üçün nəzərdə tutulmuşdur və xüsusiyyətlərin sayı çox böyük olmadıqda (<20) batch k-means-dan daha sürətli ola bilər, oxşar nəticələrlə. Təlim zamanı o, hər bir yeni instansı

tez bir zamanda cluster-a təyin etmək üçün kifayət qədər məlumat ehtiva edən bir ağac qurur, ağacda bütün instansları saxlamağa ehtiyac olmadan: bu yanaşma məhdud yaddaşdan istifadə edərək nəhəng dataset-ləri idarə etməyə imkan verir.

Mean-shift

Bu alqoritm hər bir instansın üzərində mərkəzləşmiş bir çevrə yerləşdirməklə başlayır; sonra hər bir çevrə üçün onun daxilində yerləşən bütün instansların ortalama dəyərini hesablayır və çevrəni ortalama üzərində mərkəzləşəcək şəkildə sürüşdürür. Sonra, bütün çevrələr hərəkət etməyi dayandırana qədər (yəni, hər biri onun tərkibindəki instansların ortalama dəyəri üzərində mərkəzləşənə qədər) bu mean-shift addımını təkrarlayır. Mean-shift çevrələri daha yüksək sıxlıq istiqamətində sürüşdürür, ta ki hər biri yerli sıxlıq maksimumunu tapsın. Nəhayət, çevrələri eyni yerdə (və ya kifayət qədər yaxın) qərarlaşmış bütün instanslar eyni cluster-a təyin edilir. Mean-shift DBSCAN ilə eyni bəzi xüsusiyyətlərə malikdir, məsələn, istənilən sayda və istənilən formalı cluster-ları tapa bilməsi, çox az hiperparametrə (sadəcə biri – çevrələrin radiusu, *bandwidth* adlanır) malik olması və yerli sıxlıq təxmininə əsaslanması. Lakin DBSCAN-dan fərqli olaraq, mean-shift daxili sıxlıq dəyişiklikləri olduqda cluster-ları hissələrə ayırmağa meyllidir. Təəssüf ki, onun hesablama mürəkkəbliyi $2 O(m n)$ olduğundan, böyük dataset-lər üçün uyğun deyil.

Affinity propagation

Bu alqoritmə instanslar bir-biri ilə təkrarlanan mesajlar mübadiləsi aparır, ta ki hər bir instans başqa bir instansı (və ya özünü) təmsilçi olaraq seçsin. Seçilmiş bu instanslar *exemplars* (nümunələr) adlanır. Hər

bir exemplar və onu seçən bütün instanslar bir cluster təşkil edir. Real həyat siyasətində, adətən sizə bənzər fikirləri olan bir namizədə səs vermək istəyirsiniz, eyni zamanda onların seçimi qazanmasını da istəyirsiniz, buna görə də tam razılaşmadığınız, lakin daha populyar olan bir namizəd seçə bilərsiniz. Siz adətən populyarlığı sorğular vasitəsilə qiymətləndirirsiniz. Affinity propagation oxşar şəkildə işləyir və k-means-a bənzər şəkildə cluster-ların mərkəzinə yaxın yerləşən exemplar-ları seçməyə meyllidir. Lakin k-means-dan fərqli olaraq, əvvəlcədən cluster sayını seçmək məcburiyyətində deyilsiniz: o, təlim zamanı müəyyən edilir. Üstəlik, affinity propagation müxtəlif 2 ölçülü cluster-larla yaxşı davranır. Təəssüf ki, bu alqoritmin hesablama mürəkkəbliyi $O(m)$ olduğundan, böyük dataset-lər üçün uyğun deyil.

Spectral clustering

Bu alqoritm instanslar arasındakı oxşarlıq matrisini götürür və ondan aşağı ölçülü bir embedding yaradır (yəni, matrisin ölçülülüyünü azaldır), sonra bu aşağı ölçülü fəzada başqa bir clustering alqoritmindən istifadə edir (Scikit-Learn-ın tətbiqi k-means istifadə edir). Spectral clustering mürəkkəb cluster strukturlarını tuta bilər və həmçinin qrafikləri kəsmək üçün istifadə edilə bilər (məsələn, sosial şəbəkədə dost cluster-larını müəyyən etmək üçün). O, çox sayda instansa yaxşı miqyaslanmır və cluster-lar çox fərqli ölçülərə malik olduqda yaxşı davranmır.

İndi Gaussian mixture modellərinə dərinləşək. Bunlar density estimation, clustering və anomaly detection üçün istifadə oluna bilər.

Gaussian Mixtures

Gaussian mixture model (GMM) parametrləri bilinməyən bir neçə Gaussian paylanmasının qarışığından instansların yaradıldığını fərz edən ehtimal modelidir. Tək bir Gaussian paylanmasından yaradılan bütün instanslar adətən ellipsoidə bənzəyən bir cluster təşkil edir. Hər bir cluster fərqli ellipsoidal formaya, ölçüyə, sıxlığa və istiqamətə malik ola bilər, eynilə Şəkil 9-11-də olduğu kimi. Siz bir instansı müşahidə etdikdə, bilərsiniz ki, o, Gaussian paylanmalarından birindən yaradılmışdır, lakin sizə hansından olduğu deyilmir və bu paylanmaların parametrlərini də bilmirsiniz.

GMM-in bir neçə variantı var. Ən sadə variant, `GaussianMixture` sinfində tətbiq edilmişdir, əvvəlcədən k Gaussian paylanmasının sayını bilməlisiniz. Dataset X -in aşağıdakı ehtimal prosesi vasitəsilə yaradıldığı fərz edilir:

- Hər bir instans üçün, k cluster-dan biri təsadüfi seçilir.
- j ci cluster-in seçilmə ehtimalı onun çəkisi $\phi(j)$ -dir. i ci instans üçün seçilən cluster-in indeksi $z(i)$ kimi qeyd olunur.
- Əgər i ci instans j ci cluster-a təyin edilmişsə (yəni, $z(i) = j$), onda bu instansın yeri $x(i)$ Gaussian paylanmasından təsadüfi olaraq seçilir, ortalama $\mu(j)$ və kovariasiya matrisi $\Sigma(j)$ ilə. Bu, $x(i)$ ($\mu(j)$, $\Sigma(j)$) kimi qeyd olunur.

Bəs belə bir modelə nə edə bilərsiniz? Yaxşı, dataset X verildikdə, adətən əvvəlcə çəkilər ϕ və bütün paylanma parametrləri $\mu(1)$ -dan $\mu(k)$ -yə və $\Sigma(1)$ -dan $\Sigma(k)$ -yə qədər qiymətləndirmək istəyirsiniz.

Scikit-Learn-in `GaussianMixture` sinfi bunu super asan edir:

```
from sklearn.mixture import GaussianMixture
```

```
gm = GaussianMixture(n_components=3, n_init=10)
```

```
gm.fit(X)
```

Alqoritmin qiymətləndirdiyi parametrlərə baxaq:

```
>>> gm.weights_  
array([0.39025715, 0.40007391, 0.20966893])  
>>> gm.means_  
array([[ 0.05131611,  0.07521837],  
       [-1.40763156,  1.42708225],  
       [ 3.39893794,  1.05928897]])  
>>> gm.covariances_  
array([[[ 0.68799922,  0.79606357],  
        [ 0.79606357,  1.21236106]],  
       [[ 0.63479409,  0.72970799],  
        [ 0.72970799,  1.1610351 ]],  
       [[ 1.14833585, -0.03256179],  
        [-0.03256179,  0.95490931]]])
```

Əla, yaxşı işlədi! Doğrudan da, üç cluster-dan ikisi hər biri 500 instansla, üçüncü cluster isə yalnız 250 instansla yaradılmışdı. Beləliklə, həqiqi cluster çəkili müvafiq olaraq 0.4, 0.4 və 0.2-dir və alqoritmin tapdığı da təxminən budur. Eynilə, həqiqi ortalama və kovariasiya matrisləri alqoritmin tapdıqlarına olduqca yaxındır. Bəs necə? Bu sinif *expectation-maximization* (EM - gözlənti-maksimallaşdırma) alqoritmində əsaslanır və bu, *k-means* alqoritmisi ilə çox oxşarlıqlara malikdir: o da cluster parametrlərini təsadüfi olaraq ilkinləşdirir, sonra yığılmaya qədər iki addımı təkrarlayır: əvvəlcə instansları cluster-lara təyin edir (buna *expectation step* - gözlənti addımı deyilir) və sonra cluster-ları yeniləyir (buna *maximization step* - maksimallaşdırma addımı deyilir). Tanış gəlir, elə deyilmi? Clustering kontekstində, EM-ni *k-means*-in ümumiləşdirilməsi kimi düşünə bilərsiniz, o, təkcə cluster mərkəzlərini ($\mu(1)$ -dan $\mu(k)$ -yə) tapmır, həm də onların ölçüsünü, formasını və istiqamətini ($\Sigma(1)$ -dan $\Sigma(k)$ -yə), eləcə də nisbi çəkilişlərini ($\phi(1)$ -dan $\phi(k)$)

-yə) tapır. Lakin k-means-dan fərqli olaraq, EM sərt (hard) deyil, yumşaq (soft) cluster təyinatlarından istifadə edir. Hər bir instans üçün, expectation step (gözlənti addımı) zamanı alqoritm onun hər bir cluster-a aid olma ehtimalını (cari cluster parametrlərinə əsasən) qiymətləndirir. Sonra, maximization step (maksimallaşdırma addımı) zamanı hər bir cluster dataset-dəki *bütün* instanslardan istifadə edərək yenilənir, hər bir instans onun həmin cluster-a aid olma ehtimalı ilə çəkilir. Bu ehtimallar cluster-ların instanslar üçün *responsibilities* (məsuliyyətlər) adlanır. Maximization step zamanı, hər bir cluster-ın yenilənməsi əsasən onun üçün ən çox məsuliyyət daşdığı instanslardan təsirlənəcəkdir.

XƏBƏRDARLIQ

Təəssüf ki, k-means kimi, EM də zəif həllərə yığıla bilər, buna görə də onu bir neçə dəfə işə salmaq və yalnız ən yaxşı həlli saxlamaq lazımdır. Buna görə biz `n_init`-i 10-a təyin etdik. Diqqətli olun: standart olaraq `n_init` 1-ə təyin edilir.

Alqoritmin yığılıb-yığılmadığını və neçə iterasiya çəkdiyini yoxlaya bilərsiniz:

```
>>> gm.converged_  
True  
>>> gm.n_iter_
```

```
4
```

İndi siz hər bir cluster-ın yeri, ölçüsü, forması, istiqaməti və nisbi çəkisi haqqında təxminə malik olduğunuzdan, model hər bir instansı ən çox ehtimal olunan cluster-a asanlıqla təyin edə bilər (hard clustering) və ya

onun müəyyən bir cluster-a aid olma ehtimalını qiymətləndirə bilər (soft clustering). Hard clustering üçün sadəcə `predict()` metodundan, soft clustering üçün isə `predict_proba()` metodundan istifadə edin:

```
>>> gm.predict(X)
array([0, 0, 1, ..., 2, 2, 2])
>>> gm.predict_proba(X).round(3)
array([[0.977, 0.    , 0.023],
       [0.983, 0.001, 0.016],
       [0.    , 1.    , 0.    ],
       ...,
       [0.    , 0.    , 1.    ],
       [0.    , 0.    , 1.    ],
       [0.    , 0.    , 1.    ]])
```

Gaussian mixture model *generative model* (generativ model) dir, yəni siz ondan yeni instanslar nümunə götürə bilərsiniz (qeyd edək ki, onlar cluster indeksi üzrə sıralanmışdır):

```
>>> X_new, y_new = gm.sample(6)
>>> X_new
array([[ -0.86944074, -0.32767626],
       [ 0.29836051,  0.28297011],
       [-2.8014927 , -0.09047309],
       [ 3.98203732,  1.49951491],
       [ 3.81677148,  0.53095244],
       [ 2.84104923, -0.73858639]])
>>> y_new
```

```
array([0, 0, 1, 2, 2, 2])
```

İstənilən yerdə modelin sıxlığını qiymətləndirmək də mümkündür. Bu, `score_samples()` metodundan istifadə etməklə əldə edilir: verilən hər bir instans üçün bu metod həmin yerdəki *probability density function*

(PDF)-in loqarifmini qiymətləndirir. Bal nə qədər böyükdürsə, sıxlıq bir o qədər yüksəkdir:

```
>>> gm.score_samples(X).round(2)
```

```
array([-2.61, -3.57, -3.33, ..., -3.51, -4.4 ,  
       -3.81])
```

Əgər bu balların eksponensialını hesablasanız, verilmiş instansların yerlərində PDF-in qiymətini alarsınız. Bunlar ehtimal deyil, ehtimal *sıxlıqları* dır: onlar 0 ilə 1 arasında deyil, istənilən müsbət qiymət ala bilər. Bir instansın müəyyən bir bölgəyə düşmə ehtimalını qiymətləndirmək üçün PDF-i həmin bölgə üzərində integral etməli olacaqsınız (əgər bunu bütün mümkün instans yerləri fəzası üzərində etsəniz, nəticə 1 olacaq).

Şəkil 9-16 bu modelin cluster ortalama dəyərlərini, qərar sərhədlərini (kəsikli xətlər) və sıxlıq konturlarını göstərir.

Şəkil 9-16. Öyrədilmiş Gaussian mixture modelin cluster ortalama dəyərləri, qərar sərhədləri və sıxlıq konturları

Gözəl! Alqoritm aydın şəkildə əla bir həll tapdı. Əlbəttə, biz məlumatları 2D Gaussian paylanmaları dəstindən istifadə edərək yaratmaqla onun işini asanlaşdırdıq (təəssüf ki, real həyat məlumatları həmişə o qədər Gaussian və aşağı ölçülü olmur). Biz həmçinin alqoritmə düzgün sayda cluster verdik. Çoxlu ölçü, çoxlu cluster və ya az instans olduqda, EM optimal həllə yığılmaqda çətinlik çəkə bilər. Alqoritmin öyrənməli olduğu parametrlərin sayını məhdudlaşdırmaqla tapşırığın çətinliyini azaltmaq lazım ola bilər. Bunun bir yolu, cluster-ların sahib ola biləcəyi formaların və istiqamətlərin diapazonunu məhdudlaşdırmaqdır. Buna kovariasiya matrislərinə məhdudiyyətlər qoymaqla nail olmaq olar. Bunu etmək üçün

`covariance_type` hiperparametrini aşağıdakı dəyərlərdən birinə təyin edin:

"spherical"

Bütün cluster-lar sferik olmalıdır, lakin müxtəlif diametrlərə (yəni, müxtəlif dispersiyalara) malik ola bilərlər.

"diag"

Cluster-lar istənilən ellipsoidal formada və istənilən ölçüdə ola bilər, lakin ellipsoidin oxları koordinat oxlarına paralel olmalıdır (yəni, kovariasiya matrisləri diaqonal olmalıdır).

"tied"

Bütün cluster-lar eyni ellipsoidal forma, ölçü və istiqamətə malik olmalıdır (yəni, bütün cluster-lar eyni kovariasiya matrisini paylaşır).

Standart olaraq, `covariance_type` "full"-a bərabərdir, yəni hər bir cluster istənilən forma, ölçü və istiqamətə malik ola bilər (onun öz məhdudiyyətsiz kovariasiya matrisi var). Şəkil 9-17 `covariance_type` "tied" və ya "spherical" olaraq təyin edildikdə EM alqoritminin tapdığı həlləri təsvir edir.

Şəkil 9-17. Tied cluster-lar (sol) və spherical cluster-lar (sağ) üçün Gaussian mixtures

QEYD

GaussianMixture modelinin təliminin hesablama mürəkkəbliyi instansların sayı m , ölçülərin sayı n , cluster-ların sayı k və kovariasiya matrislərinə qoyulan məhdudiyyətlərdən asılıdır. Əgər

`covariance_type` "spherical" və ya "diag" olarsa, məlumatların clustering strukturuna malik olduğu fərz edilərsə, $O(kmn)$ təşkil edir. Əgər `covariance_type` "tied" və ya "full" olarsa, $O(_kmn_2 + _kn_3)$ təşkil edir, buna görə də çox sayda xüsusiyyətə malik dataset-lərə yaxşı miqyaslanmayacaq.

Gaussian mixture modelləri həmçinin anomaly detection üçün də istifadə edilə bilər. Bunu növbəti bölmədə görəcəyik.

Gaussian Mixtures-in Anomaly Detection üçün istifadəsi

Anomaly detection üçün Gaussian mixture modelindən istifadə olduqca sadədir: aşağı sıxlıqlı bölgədə yerləşən istənilən instans anomaly hesab edilə bilər. Hansı sıxlıq həddini istifadə edəcəyinizi müəyyən etməlisiniz. Məsələn, qüsurlu məhsulları aşkar etməyə çalışan bir istehsal şirkətində, qüsurlu məhsulların nisbəti adətən yaxşı məlumdur. Tutaq ki, bu, 2%-ə bərabərdir. Siz sıxlıq həddini, instansların 2%-nin həmin həddən aşağı sıxlıqlı bölgələrdə yerləşməsinə səbəb olan qiymətə təyin edirsiniz. Əgər çox sayda false positive (yəni, tamamilə yaxşı məhsulların qüsurlu kimi işarələnməsi) aldıığınızı görsəniz, həddi aşağı sala bilərsiniz. Əksinə, çox sayda false negative (yəni, sistemin qüsurlu kimi işarələmədiyi qüsurlu məhsullar) varsa, həddi artırma bilərsiniz. Bu, adi precision/recall mübadiləsidir (bax Fəsil 3). Aşağıdakı kod dördüncü persentil ən aşağı sıxlığı hədd kimi istifadə edərək outlier-ləri necə müəyyən edəcəyinizi göstərir (yəni, instansların təxminən 4%-i anomaly kimi işarələnəcək):

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 2)

anomalies = X[densities < density_threshold]
```


Şəkil 9-18 bu anomalies-ləri ulduzlar kimi təsvir edir.

Sıx əlaqəli bir tapşırıq *novelty detection* (yenilik aşkarlanması) dır: o, anomaly detection-dan fərqlənir ki, alqoritmin "təmiz" bir dataset üzərində, outlier-lərlə çirklənməmiş şəkildə öyrədildiyi fərz edilir, halbuki anomaly detection bu fərziyyəni irəli sürmür. Həqiqətən də, outlier detection çox vaxt dataset-i təmizləmək üçün istifadə olunur.

İPUCU

Gaussian mixture modelləri outlier-lər də daxil olmaqla bütün məlumatları uyğunlaşdırmağa çalışır; əgər onlardan çoxunuz varsa, bu, modelin "normallıq" haqqında təsəvvürünü təhrif edə bilər və bəzi outlier-lər səhvən normal hesab edilə bilər. Bu baş verərsə, modeli bir dəfə uyğunlaşdırmağa, ondan istifadə edərək ən ekstremal outlier-ləri aşkar edib silməyə, sonra təmizlənmiş dataset-də modeli yenidən uyğunlaşdırmağa cəhd edə bilərsiniz. Digər bir yanaşma, robust covariance estimation metodlarından istifadə etməkdir ([EllipticEnvelope](#) sinfinə baxın).

Şəkil 9-18. Gaussian mixture model istifadə edərək anomaly detection

k-means kimi, GaussianMixture alqoritmi də cluster-ların sayını təyin etməyi tələb edir. Bəs bu sayı necə tapa bilərsiniz?

Cluster Sayının Seçilməsi

k-means ilə, müvafiq cluster sayını seçmək üçün inertia və ya silhouette score-dan istifadə edə bilərsiniz. Lakin Gaussian mixtures ilə bu metrikalardan istifadə etmək mümkün deyil, çünki cluster-lar sferik deyilsə və ya müxtəlif ölçülərə malikdirsə, onlar etibarlı deyil. Bunun

əvəzinə, *theoretical information criterion* (nəzəri informasiya meyarı) kimi *Bayesian information criterion* (BIC) və ya *Akaike information criterion* (AIC)-ni minimuma endirən modeli tapmağa cəhd edə bilərsiniz. Bunlar Tənlik 9-1-də müəyyən edilmişdir.

Tənlik 9-1. Bayesian information criterion (BIC) və Akaike information criterion (AIC)

$$BIC = \log(m) p - 2 \log(L^*)$$

$$AIC = 2p - 2 \log(L^*)$$

Bu tənliklərdə:

m həmişə olduğu kimi instansların sayıdır.

p model tərəfindən öyrənilən parametrlərin sayıdır.

L^* modelin *likelihood function* (ehtimal funksiyası) funksiyasının maksimallaşdırılmış dəyəridir.

Həm BIC, həm də AIC daha çox parametr öyrənən modelləri (məsələn, daha çox cluster) cəzalandırır və məlumatlara yaxşı uyğunlaşan modelləri mükafatlandırır. Onlar tez-tez eyni modeli seçirlər.

Fərqləndikdə, BIC tərəfindən seçilən model AIC tərəfindən seçiləndən daha sadə (daha az parametr) olmağa meyllidir, lakin məlumatlara o qədər də yaxşı uyğunlaşmır (bu, xüsusilə daha böyük dataset-lər üçün doğrudur).

LİKELİHOOD FUNKSİYASI

"Ehtimal" (probability) və "likelihood" (ehtimal) terminləri gündəlik dildə çox vaxt bir-birini əvəz edən kimi istifadə olunur, lakin statistikada onlar çox fərqli mənalara malikdir. Bəzi parametrləri θ olan bir statistik model verildikdə, "probability" sözü gələcək bir nəticənin nə qədər mümkün

olduğunu təsvir etmək üçün (parametr dəyərlərini θ bilərək) istifadə olunur, "likelihood" sözü isə nəticə x məlum olduqdan sonra müəyyən bir parametr dəyərləri dəstinin θ nə qədər mümkün olduğunu təsvir etmək üçün istifadə olunur.

-4 və +1 mərkəzlərində yerləşən iki Gaussian paylanmasının 1D qarışıq modelini nəzərdən keçirək. Sadəlik üçün, bu oyuncaq modelin hər iki paylanmanın standart sapmalarını idarə edən tək bir θ parametri var. Şəkil 9-19-un yuxarı-sol kontur qrafiki bütün modeli $f(x; \theta)$ həm x , həm də θ funksiyası kimi göstərir. Gələcək nəticənin x ehtimal paylanmasını qiymətləndirmək üçün model parametrini θ təyin etməlisiniz. Məsələn, əgər θ -nı 1.3-ə təyin etsəniz (horizontal xətt), aşağı-sol qrafikdə göstərilən probability density function $f(x; \theta = 1.3)$ alırsınız. Tutaq ki, x -in -2 və +2 arasına düşmə ehtimalını qiymətləndirmək istəyirsiniz. Bu diapazonda PDF-in inteqralını hesablamalısınız (yəni, kölgəli bölgənin sahəsi). Bəs əgər θ -nı bilmirsinizsə və əvəzində tək bir instans $x = 2.5$ müşahidə etmişinizsə (yuxarı-sol qrafikdəki şaquli xətt)? Bu halda, likelihood funksiyasını $\mathcal{L}(\theta | x = 2.5) = f(x = 2.5; \theta)$ alırsınız, yuxarı-sağ qrafikdə təsvir edilmişdir.

Şəkil 9-19. Modelin parametrik funksiyası (yuxarı sol) və bəzi törəmə funksiyalar: PDF (aşağı sol), likelihood funksiyası (yuxarı sağ) və log likelihood funksiyası (aşağı sağ)

Qısacası, PDF x -in funksiyasıdır (θ sabit olmaqla), likelihood funksiyası isə θ -nın funksiyasıdır (x sabit olmaqla). Anlamaq vacibdir ki, likelihood funksiyası ehtimal paylanması deyil: əgər siz ehtimal paylanmasını x -in bütün mümkün dəyərləri üzərində inteqral etsəniz, həmişə 1 alırsınız,

lakin likelihood funksiyasını θ -nın bütün mümkün dəyərləri üzərində integral etsəniz, nəticə istənilən müsbət qiymət ola bilər.

Dataset X verildikdə, ümumi bir tapşırıq model parametrləri üçün ən çox ehtimal olunan dəyərləri qiymətləndirməyə çalışmaqdır. Bunu etmək üçün, X verildikdə likelihood funksiyasını maksimallaşdıran dəyərləri tapmalısınız. Bu nümunədə, əgər siz tək bir instans $x = 2.5$ müşahidə etmisinizsə, θ -nın *maximum likelihood estimate* (MLE) $\theta^* = 1.5$ -dir. Əgər θ üzərində əvvəlki ehtimal paylanması g mövcuddursa, onu nəzərə alaraq yalnız $\mathcal{L}(\theta | x)$ -i maksimallaşdırmaq əvəzinə $\mathcal{L}(\theta | x)g(\theta)$ -ni maksimallaşdırmaq mümkündür. Buna *maximum a-posteriori* (MAP) estimation deyilir. MAP parametr dəyərlərini məhdudlaşdırdığı üçün onu MLE-nin regularized (nizamlanmış) versiyası kimi düşünə bilərsiniz.

Diqqət yetirin ki, likelihood funksiyasını maksimallaşdırmaq onun loqarifmini maksimallaşdırmağa bərabərdir (Şəkil 9-19-un aşağı-sağ qrafikində təsvir edilmişdir). Həqiqətən də, loqarifm ciddi artan bir funksiyadır, buna görə də əgər θ log likelihood-i maksimallaşdırırsa, o, eyni zamanda likelihood-i də maksimallaşdırır. Məlum olur ki, ümumiyyətlə log likelihood-i maksimallaşdırmaq daha asandır. Məsələn, əgər siz bir neçə müstəqil instans $x(1)$ -dan $x(m)$ -yə qədər müşahidə etmisinizsə, fərdi likelihood funksiyalarının hasilini maksimallaşdıran θ dəyərini tapmalısınız. Lakin loqarifmin hasilləri cəmlərə çevirmək sehrli xüsusiyyəti sayəsində ($\log(ab) = \log(a) + \log(b)$), log likelihood funksiyalarının hasilini (hasil deyil) maksimallaşdırmaq daha sadədir.

Siz θ^* -ni – likelihood funksiyasını maksimallaşdıran θ dəyərini – qiymətləndirdikdən sonra, $L^* = L(\theta^*, X)$ hesablamağa hazırsınız. Bu, AIC və BIC-i hesablamaq üçün istifadə olunan dəyərdir; siz onu modelin

məlumatlara nə qədər yaxşı uyğunlaşdığının ölçüsü kimi düşünə bilərsiniz.

BIC və AIC-i hesablamaq üçün `bic()` və `aic()` metodlarını çağırın:

```
>>> gm.bic(X)
8189.747000497186
>>> gm.aic(X)
```

```
8102.521720382148
```

Şəkil 9-20 müxtəlif cluster sayları k üçün BIC-i göstərir. Gördüyünüz kimi, həm BIC, həm də AIC $k=3$ olduqda ən aşağıdır, buna görə də bu, çox güman ki, ən yaxşı seçimdir.

Şəkil 9-20. Müxtəlif k cluster sayları üçün AIC və BIC

Bayesian Gaussian Mixture Modelləri

Optimal cluster sayını əl ilə axtarmaq əvəzinə,

`BayesianGaussianMixture` sinfindən istifadə edə bilərsiniz. Bu sinif lazımsız cluster-lara sıfıra bərabər (və ya sıfıra yaxın) çəkilər verməyə qadirdir. `n_components` cluster sayını, optimal cluster sayından böyük olduğuna inanmaq üçün əsasınız olan bir dəyəərə təyin edin (bu, problem haqqında bəzi minimal bilikləri fərz edir) və alqoritm lazımsız cluster-ları avtomatik olaraq aradan qaldıracaq. Məsələn, cluster sayını 10-a təyin edək və nə baş verdiyinə baxaq:

```
>>> from sklearn.mixture import BayesianGaussianMixture

>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10,
random_state=42)
>>> bgm.fit(X)
```

```
>>> bgm.weights_.round(2)
```

```
array([0.4 , 0.21, 0.4 , 0. , 0. , 0. , 0. , 0. ,  
       , 0. , 0. ])
```

Mükəmməl: alqoritm avtomatik olaraq yalnız üç cluster-ın lazım olduğunu aşkar etdi və nəticədə cluster-lar demək olar ki, Şəkil 9-16-dakılarla eynidir.

Gaussian mixture modelləri haqqında son bir qeyd: onlar ellipsoidal formalı cluster-larda əla işləsələr də, çox fərqli formalı cluster-larla o qədər də yaxşı işləmirlər. Məsələn, gəlin görək moons dataset-ni clustering etmək üçün Bayesian Gaussian mixture modelindən istifadə etsək nə baş verir (bax Şəkil 9-21).

Oups! Alqoritm ümitsizcə ellipsoidlər axtardı, buna görə də iki yerinə səkkiz müxtəlif cluster tapdı. Sıxlıq təxmini o qədər də pis deyil, buna görə də bu model bəlkə də anomaly detection üçün istifadə edilə bilər, lakin o, iki ayı (moons) müəyyən edə bilmədi. Bu fəslə yekunlaşdırmaq üçün gəlin özbaşına formalı cluster-larla məşğul ola bilən bir neçə alqoritmə qısaca nəzər salaq.

Şəkil 9-21. Qeyri-ellipsoidal cluster-lara Gaussian mixture uyğunlaşdırmaq

Anomaly və Novelty Detection üçün Digər Alqoritmlər

Scikit-Learn anomaly detection və ya novelty detection üçün nəzərdə tutulmuş digər alqoritmləri də tətbiq edir:

Fast-MCD (minimum covariance determinant)

`EllipticEnvelope` sinfi tərəfindən tətbiq edilən bu alqoritm, xüsusən dataset-i təmizləmək üçün outlier detection üçün faydalıdır. O, normal instansların (inliers) tək bir Gaussian paylanmasından (qarışıq deyil) yaradıldığını fərz edir. O, həmçinin dataset-in bu Gaussian paylanmasından yaradılmamış outlier-lərlə çirkləndiyini fərz edir. Alqoritm Gaussian paylanmasının parametrlərini (yəni, inliers ətrafındakı elliptik zərfin formasını) qiymətləndirərkən, çox güman ki, outlier olan instansları nəzərə almamağa diqqət edir. Bu texnika elliptik zərfin daha yaxşı qiymətləndirilməsini verir və beləliklə, alqoritmi outlier-ləri müəyyən etməkdə daha yaxşı edir.

Isolation forest

Bu, xüsusilə yüksək ölçülü dataset-lərdə outlier detection üçün səmərəli bir alqoritmdir. Alqoritm təsadüfi bir random forest qurur, burada hər bir qərar ağacı təsadüfi olaraq böyüdülmür: hər bir nodda o, təsadüfi bir xüsusiyyət seçir, sonra dataset-i iki hissəyə bölmək üçün təsadüfi bir hədd dəyəri (minimum və maksimum dəyərlər arasında) seçir. Dataset bu şəkildə tədricən parçalanır, ta ki bütün instanslar digər instanslardan təcrid olunana qədər. Anomalies adətən digər instanslardan uzaq olur, buna görə də orta hesabla (bütün qərar ağacları üzərində) onlar normal instanslara nisbətən daha az addımda təcrid olunmağa meyllidirlər.

Local outlier factor (LOF)

Bu alqoritm də outlier detection üçün yaxşıdır. O, müəyyən bir instans ətrafındakı instansların sıxlığını onun qonşularının ətrafındakı sıxlıqla müqayisə edir. Bir anomaly çox vaxt onun k -ən yaxın qonşularından daha təcrid olmuşdur.

One-class SVM

Bu alqoritm novelty detection üçün daha uyğundur. Xatırlayın ki, kernelized SVM classifier iki sinifi ayırmaq üçün əvvəlcə bütün instansları (dolayısı ilə) yüksək ölçülü fəzaya xəritələşdirir, sonra bu yüksək ölçülü fəzada linear SVM classifier istifadə edərək iki sinifi ayırır (bax Fəsil 5). Bizim sadəcə bir sinif instansımız olduğundan, one-class SVM alqoritm əvəzində instansları yüksək ölçülü fəzada mənşədən ayırmağa çalışır. Orijinal fəzada, bu, bütün instansları əhatə edən kiçik bir bölgə tapmağa uyğun olacaq. Əgər yeni bir instans bu bölgəyə düşmürsə, o, anomaly-dir. Tənzimləmək üçün bir neçə hiperparametr var: kernelized SVM üçün adi olanlar, üstəlik yeni bir instansın əslində normal olduğu halda səhvən yeni (novel) hesab edilməsi ehtimalına uyğun gələn bir margin hiperparametri. O, xüsusilə yüksək ölçülü dataset-lərlə əla işləyir, lakin bütün SVM-lər kimi böyük dataset-lərə yaxşı miqyaslanmır.

PCA və `inverse_transform()` metoduna malik digər dimensionality reduction texnikaları

Əgər normal bir instansın rekonstruksiya səhvini (reconstruction error) bir anomaly-nın rekonstruksiya səhvi ilə müqayisə etsəniz, sonuncu adətən çox daha böyük olacaq. Bu, sadə və çox vaxt olduqca səmərəli bir anomaly detection yanaşmasıdır (bu fəslin məşqlərində bir nümunəyə baxın).

Məşqlər

1. Clustering-i necə tərif edərdiniz? Bir neçə clustering alqoritmının adını çəkkə bilərsinizmi?

2. Clustering alqoritmlərinin əsas tətbiq sahələrindən bəziləri hansılardır?
3. k-means istifadə edərkən düzgün cluster sayını seçmək üçün iki texnikanı təsvir edin.
4. Label propagation nədir? Onu niyə və necə həyata keçirərdiniz?
5. Böyük dataset-lərə miqyaslına bilən iki clustering alqoritminin adını çəkə bilərsinizmi? Bəs yüksək sıxlıqlı bölgələri axtaran ikisinin?
6. Active learning-in faydalı olacağı bir istifadə halı düşünə bilərsinizmi? Onu necə həyata keçirərdiniz?
7. Anomaly detection və novelty detection arasındakı fərq nədir?
8. Gaussian mixture nədir? Onu hansı tapşırıqlar üçün istifadə edə bilərsiniz?
9. Gaussian mixture modelindən istifadə edərkən düzgün cluster sayını tapmaq üçün iki texnikanın adını çəkə bilərsinizmi?
10. Klassik Olivetti faces dataset-i 400 bozqanadlı 64×64 piksel üz şəklini ehtiva edir. Hər bir şəkil 4,096 ölçülü 1D vektora düzəldilir. Qırx müxtəlif insan (hər biri 10 dəfə) fotosəkli çəkilmişdir və adi tapşırıq hər bir şəkildə hansı insanın təmsil olunduğunu proqnozlaşdırma bilən bir model öyrətməkdir. Dataset-i `sklearn.datasets.fetch_olivetti_faces()` funksiyasından istifadə edərək yükləyin, sonra onu training set, validation set və test set-ə bölün (qeyd edək ki, dataset artıq 0 ilə 1 arasında miqyaslandırılıb). Dataset olduqca kiçik olduğundan, hər bir setdə hər bir insan üçün eyni sayda şəkil olmasını təmin etmək üçün ehtimal ki, stratified sampling istifadə etmək istəyəcəksiniz. Sonra, şəkilləri k-means istifadə edərək clustering edin və yaxşı sayda cluster-ə malik olduğunuzdan əmin olun (bu fəsildə müzakirə edilən texnikalardan birini istifadə edərək). Cluster-ları vizuallaşdırın: hər bir cluster-da oxşar üzlər görürsünüzmü?
11. Olivetti faces dataset-i ilə davam edərək, hər bir şəkildə hansı insanın təmsil olunduğunu proqnozlaşdırmaq üçün bir classifier öyrədin və onu validation set-də qiymətləndirin. Sonra, dimensionality reduction vasitəsi kimi k-means istifadə edin və azaldılmış set üzərində bir classifier öyrədin. Classifier-in ən yaxşı performans əldə etməsinə imkan verən cluster sayını axtarın: hansı performansla çata bilərsiniz? Bəs azaldılmış setdən gələn xüsusiyyətləri orijinal xüsusiyyətlərə əlavə etsəniz nə olar (yenə ən yaxşı cluster sayını axtararaq)?
12. Olivetti faces dataset-i üzərində bir Gaussian mixture modeli öyrədin. Alqoritmi sürətləndirmək üçün, yəqin ki, dataset-in

ölçülülüyünü azaltmalısınız (məsələn, PCA istifadə edərək, dispersiyanın 99%-ni qoruyaraq). Modeldən istifadə edərək bəzi yeni üzlər yaradın (`sample()` metodundan istifadə edərək) və onları vizuallaşdırın (əgər PCA istifadə etmisinizsə, onun `inverse_transform()` metodundan istifadə etməli olacaqsınız). Bəzi şəkilləri modifikasiya etməyə çalışın (məsələn, fırlatmaq, çevirmək, qaraltmaq) və modelin anomalies-ləri aşkar edə biləcəyinə baxın (yəni, normal şəkillər və anomalies-lər üçün `score_samples()` metodunun çıxışını müqayisə edin).

13. Bəzi dimensionality reduction texnikaları həmçinin anomaly detection üçün istifadə edilə bilər. Məsələn, Olivetti faces dataset-ni götürün və onu PCA ilə azaldın, dispersiyanın 99%-ni qoruyaraq. Sonra hər bir şəkil üçün rekonstruksiya səhvini hesablayın. Sonra, əvvəlki məşqdə qurduğunuz bəzi modifikasiya edilmiş şəkilləri götürün və onların rekonstruksiya səhvinə baxın: onun nə qədər böyük olduğuna diqqət yetirin. Əgər rekonstruksiya edilmiş bir şəkli çəksəniz, səbəbini görərsiniz: o, normal bir üzü rekonstruksiya etməyə çalışır.

Bu məşqlərin həlləri bu fəslin notebook-unun sonunda,

<https://homl.info/colab3> ünvanında mövcuddur.